

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_ARGS 64
#define BUF_SIZE 1024

// Utility function to trim leading and trailing spaces
char *trim_spaces(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

void execute_redirection(char *line) {
    char *stdout_file = NULL;
    char *stderr_file = NULL;
    int stdout_append = 0;
    int stderr_append = 0;

    // 1. Parse stderr Redirection (2>> or 2>)
    char *err_append_pos = strstr(line, "2>>");
    char *err_trunc_pos = strstr(line, "2>");

    if (err_append_pos) {
        stderr_append = 1;
        *err_append_pos = '\0';
        stderr_file = trim_spaces(err_append_pos + 3);
    } else if (err_trunc_pos) {
        stderr_append = 0;
        *err_trunc_pos = '\0';
    }
}
```

```

        stderr_file = trim_spaces(err_trunc_pos + 2);
    }

    // 2. Parse stdout Append / Overwrite Redirection (>> or >)
    char *out_append_pos = strstr(line, ">>");
    char *out_trunc_pos = strchr(line, '>');

    if (out_append_pos) {
        stdout_append = 1;
        *out_append_pos = '\0';
        stdout_file = trim_spaces(out_append_pos + 2);
    } else if (out_trunc_pos) {
        stdout_append = 0;
        *out_trunc_pos = '\0';
        stdout_file = trim_spaces(out_trunc_pos + 1);
    }

    // Clean base command token string
    char *cmd_str = trim_spaces(line);

    char *args[MAX_ARGS];
    int arg_count = 0;
    char *token = strtok(cmd_str, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
    args[arg_count] = NULL;

    if (arg_count == 0) return;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return;
    } else if (pid == 0) {

```

```

// Child Process: Redirect stdout if requested
if (stdout_file && strlen(stdout_file) > 0) {
    int flags = O_WRONLY | O_CREAT | (stdout_append ? O_APPEND : O_TRUNC);
    int out_fd = open(stdout_file, flags, 0644);
    if (out_fd < 0) {
        perror("Failed to open stdout redirection file");
        exit(EXIT_FAILURE);
    }
    dup2(out_fd, STDOUT_FILENO);
    close(out_fd);
}

// Child Process: Redirect stderr if requested
if (stderr_file && strlen(stderr_file) > 0) {
    int flags = O_WRONLY | O_CREAT | (stderr_append ? O_APPEND : O_TRUNC);
    int err_fd = open(stderr_file, flags, 0644);
    if (err_fd < 0) {
        perror("Failed to open stderr redirection file");
        exit(EXIT_FAILURE);
    }
    dup2(err_fd, STDERR_FILENO);
    close(err_fd);
}

execvp(args[0], args);
perror("Execution failed");
exit(EXIT_FAILURE);
} else {
    // Parent Process: Wait for process completion
    int status;
    waitpid(pid, &status, 0);
}
}

int main() {
    char input_buf[BUF_SIZE];

```

```

    while (1) {
        printf("\nAppend/Stderr Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Remove newline

        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_redirection(input_buf);
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano append_stderr.c
abhis@MIKHAIL-37:~$ gcc -g -Wall append_stderr.c -o append_stderr
abhis@MIKHAIL-37:~$ ./append_stderr

Append/Stderr Shell > echo "First Entry" > log.txt

Append/Stderr Shell > echo "Second Entry" >> log.txt

Append/Stderr Shell > cat log.txt
"First Entry"
"Second Entry"

Append/Stderr Shell > ls /nonexistent_folder_xyz 2> error.log

Append/Stderr Shell > cat error.log
ls: cannot access '/nonexistent_folder_xyz': No such file or directory

Append/Stderr Shell > cat /invalid_file_123 2>> error.log

Append/Stderr Shell > cat error.log
ls: cannot access '/nonexistent_folder_xyz': No such file or directory
cat: /invalid_file_123: No such file or directory

Append/Stderr Shell > ls /bin /invalid_dir > stdout.log 2> stderr.log

Append/Stderr Shell > exit
```

```
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./append_stderr
==771== Memcheck, a memory error detector
==771== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==771== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==771== Command: ./append_stderr
==771==

Append/Stderr Shell > exit
==771==
==771== HEAP SUMMARY:
==771==    in use at exit: 0 bytes in 0 blocks
==771==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==771==
==771== All heap blocks were freed -- no leaks are possible
==771==
==771== For lists of detected and suppressed errors, rerun with: -s
==771== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```